LAB 1: INTRODUCTION TO MATLAB

```
%--------------------------------------
% Fill in your student Name and ID.
Students.Name = '';
Students.ID = '';
%--------------------------------------
```

0. Objectives

This lab is divided into two parts. The first part is an introduction to some useful functions in **MATLAB** and how they will be used in the series of following labs. The second part is an application of how to calculate Laplace and inverse Laplace using **MATLAB**. Each part has questions at the end that need to be completed.

1. How to Submit Lab Reports

A **MATLAB** LiveScript is an interactive document that combines **MATLAB** code with embedded output, formatted text, equations, and images in a single environment called the Live Editor. LiveScripts are stored using the LiveScript file format in a file with a `.mlx` extension.

Labs consist of a **MATLAB** LiveScript file. You should read and answer the questions inside the LiveScript document. At the end of the lab:

1. Fill your `Name`, and `ID` at the header of the LiveScript.
2. Ensure that the LiveScript document runs without errors.
3. Export the LiveScript to a PDF format (Click on the arrow under the save icon).
4. Submit the LiveScript and the generated PDF in myCourses.
5. If requested, attach any auto-generated MAT-file (explained in section 2.4).

2. Useful Functions

2.1. Documentation in **MATLAB**

All the functions in **MATLAB** are accompanied with detailed documentation about their usage. You can access inline documentation by typing **help** followed by the function name. For example

```
help abs
```

```
 ABS    Absolute value.
    ABS(X) is the absolute value of the elements of X. When
    X is complex, ABS(X) is the complex modulus (magnitude) of
    the elements of X.
```

```
    See also SIGN, ANGLE, UNWRAP, HYPOT.

    Documentation for abs

       doc abs

    Other functions named abs

       codistributed/abs    duration/abs    iddata/abs    sym/abs

       dlarray/abs          gpuArray/abs
```

A more complete documentation can be also obtained by using the **doc** command. For example

```
doc abs
```

2.2. Arrays in MATLAB

Matrices and arrays are the fundamental representation of information and data in **MATLAB**. For a basic overview of matrix and array manipulation, you can refer to **MATLAB** on-line documentation: Matrices and Arrays, and if needed watch the following video Working with Arrays in MATLAB.

Arrays will be used to model a discrete-time signal. An example of a discrete-time signal is:

```
dt = 0.01; % timestep of discrete signal in seconds
T = 10; % Final value of the time signal in seconds

% This creates an array |t = [0, 0.01, 0.02, 0.03, ..., 9.98, 9.99, 10.0]|.
t = 0:dt:T; % time signal
```

A sine wave signal of amplitude 1 and period 1 rad/second

$$x_1(t) = \sin(2\pi t)$$

can be written as

```
x1 = sin(2*pi*t);
```

Logical operations can be used to construct a desired signal. For example, the following signal

$$x_2(t) = \begin{cases} t, & 0 \leqslant t < 5 \\ -t + 10, & t \geqslant 5 \end{cases}$$

Can be written as

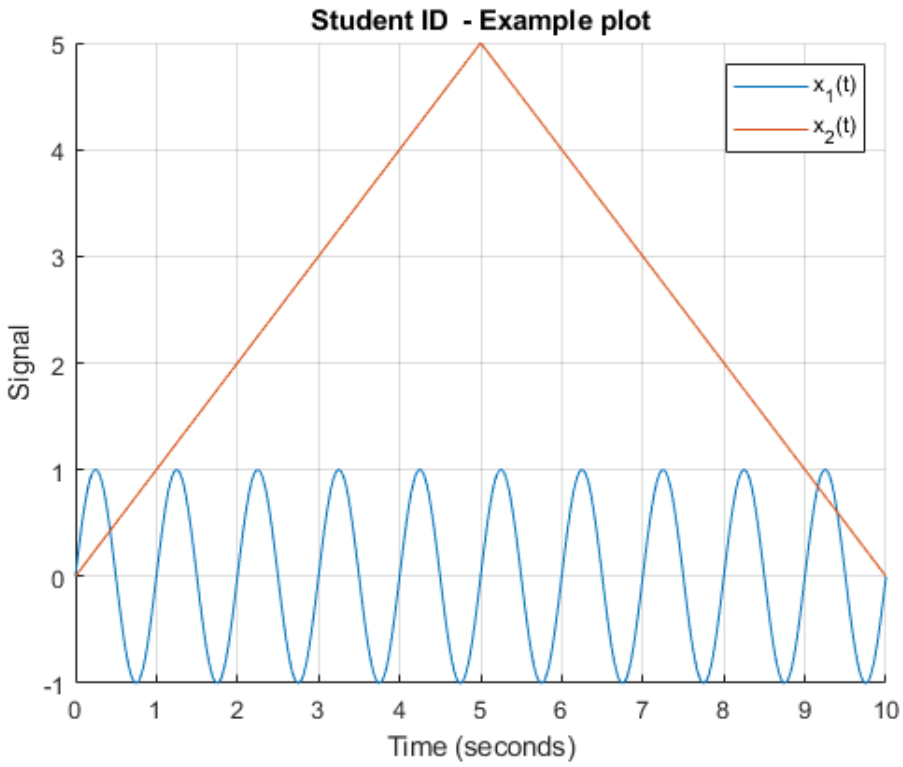
```
x2 = zeros(size(t)); % create a zero signal with the same size as time
x2(t < 5) = t(t < 5); % value is equal to time, but only when time < 5
x2(t >= 5) = -t(t >= 5) + 10; % value is equal to -time+10, but only when time >= 5
```

Other useful functions are: **size**, **length**, **ones**. You can use **MATLAB** help for more information.

2.3. Plotting signals

In order to present results you can use the **`plot`** function. **`plot(X,Y)`** creates a 2-D line plot of the data in Y versus the corresponding values in X.

```
figure;
hold on;
grid on;
plot(t, x1);
plot(t, x2);
title(sprintf('Student ID %s - Example plot', Students.ID));
xlabel('Time (seconds)')
ylabel('Signal')
legend('x_1(t)', 'x_2(t)')
hold off;
```



2.4. Matrices in MATLAB

Matrices are extensions of arrays. They will be used to represent linear systems in a state space form.

For example to represent the matrix A:

$$A = \begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$$

We write

```
A = [8, 1, 6; 3, 5, 7; 4, 9, 2]
```

```
A =

     8     1     6

     3     5     7

     4     9     2
```

Determinant of A can be obtained by:

```
det(A)
```

```
ans =

  -360
```

Eigenvalues of A can be obtained by:

```
eig(A)
```

```
ans =

   15.0000

    4.8990

   -4.8990
```

Inverse of A can be obtained by:

```
inv(A)
```

```
ans =

    0.1472   -0.1444    0.0639

   -0.0611    0.0222    0.1056

   -0.0194    0.1889   -0.1028
```

2.5. Saving/loading data

In **MATLAB** you can save any or all the variables in the current workspace to a MAT-file (.mat). You can then reuse the workspace variables later during the current MATLAB session or during another session by loading the saved MAT-file.

For example, to save a variable, you can use:

```
example = sprintf('A variable generated by Student ID <%s>', Students.ID);
save('Lab01.mat', 'example')
```

Now we remove the variable **example** from the workspace.

```
clear example
```

You can recover the variable **example** to the workspace from the MAT-file, by doing

```
load('Lab01.mat', 'example')
display(example)
```

```
example =

    'A variable generated by Student ID <>'
```

For hardware labs (labs that requires the use of external hardware), it is possible to save result of your manipulation in this manner. The resulting file can be used to re-examine your data, or change your plots.

Question 1 (4 marks)

Answer the following questions in the LiveScript. (Observe that if you finish an expression with a semicolon the results will not show in your report. Use semicolons at the end of your expression with the intention to show or hide your answer)

- Define a time signal from 0 to 10 seconds, with a time step of 0.1 second.

```
t = [];
```

- Define a unit signal equals to 1 from 0 to 10 second. (The unit signal should have the same size as time)

$$y_1(t) = 1, \quad t \geqslant 0$$

```
y1 = [];
```

- Define a delayed signal equals zero from 0 to 5 seconds and to -1 from 5 to 10 seconds.

$$y_2(t) = \begin{cases} 0, & t < 5 \\ -1, & t \geqslant 5 \end{cases}$$

```
y2 = [];
```

- Define a ramp signal with slope equals 0.1.

$$y_3(t) = 0.1t, \quad t \geqslant 0$$

```
y3 = [];
```

- Define a decaying exponential signal with time constant of 10 second.

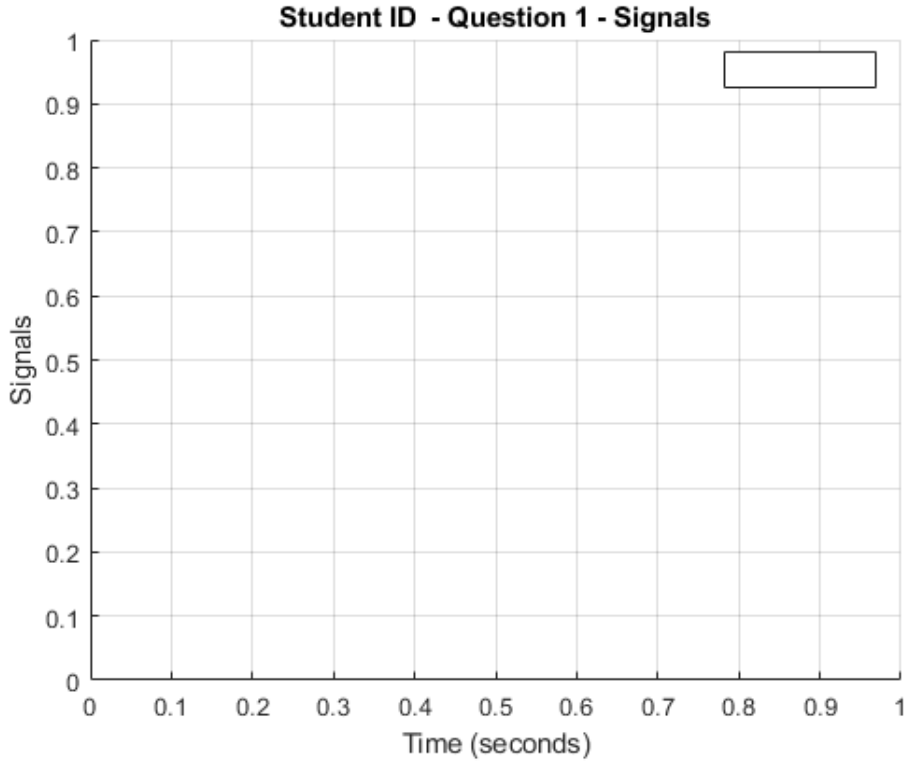
$$y_4(t) = e^{-\frac{t}{10}}$$

```
y4 = [];
```

- Plot in the same graph the signals ($y_1(t)$, $y_2(t)$, $y_3(t)$, and $y_4(t)$) versus the time signal.

```
figure;
hold on
% Fill in your plots command

title(sprintf('Student ID %s - Question 1 - Signals', Students.ID));
xlabel('Time (seconds)')
ylabel('Signals')
grid on;
legend('y_1(t)', 'y_2(t)', 'y_3(t)', 'y_4(t)', 'location', 'best')
```



3. Laplace transforms

Recall that the (one-sided) Laplace transform of a function $f(t)$ is defined as:

$$F(s) = \int_{0-}^{+\infty} f(t)e^{-st}dt$$

In this section, we will learn how to compute the Laplace and inverse Laplace transforms using **MATLAB**.

3.1. Computing Laplace Transform

The command **laplace** in **MATLAB** provides an easy way to calculate the Laplace transform $F(s)$ of a function $f(t)$. To use **laplace**, first you need to specify that the variable t and s are symbolic ones. This is done with the command:

```
syms t s
```

Then, we define the function $f(t)$.

```
f = exp(-t)
```

```
f =

exp(-t)
```

Finally, we compute the Laplace transform using the function `laplace`

```
F = laplace(f, t, s)
```

```
F =

1/(s + 1)
```

To get more information about the **laplace** function, type:

```
help laplace
```

```
--- help for sym/laplace ---
 LAPLACE Laplace transform.
    L = LAPLACE(F) is the Laplace transform of the sym F with default
    independent variable t.  The default return is a function of s.
    If F = F(s), then LAPLACE returns a function of z:  L = L(z).
    By definition, L(s) = int(F(t)*exp(-s*t),t,0,inf).

    L = LAPLACE(F,z) makes L a function of z instead of the default s:
    LAPLACE(F,z) <=> L(z) = int(F(t)*exp(-z*t),t,0,inf).
```

```
    L = LAPLACE(F,w,u) makes L a function of u instead of the
    default s (integration with respect to w).
    LAPLACE(F,w,u) <==> L(u) = int(F(w)*exp(-u*w),w,0,inf).

    Examples:
     syms a s t w x F(t)
     laplace(t^5)          returns   120/s^6
     laplace(exp(a*s))     returns   -1/(a-z)
     laplace(sin(w*x),t)   returns   w/(t^2+w^2)
     laplace(cos(x*w),w,t) returns   t/(t^2+x^2)
     laplace(x^(3/2),t)    returns   (3*pi^(1/2))/(4*t^(5/2))
     laplace(diff(F(t)))   returns   s*laplace(F(t),t,s) - F(0)

    See also SYM/ILAPLACE, SYM/FOURIER, SYM/HTRANS, SYM/ZTRANS, SUBS.
```

Often, using **`laplace`** function gives an expression that is mathematically correct, but not in its simplest form. In such cases, the function `simplify` can be used to simplify the expression. For example:

```
simplify(2/(s + 1)-s/(s + 1)^2)
```

```
ans =

(s + 2)/(s + 1)^2
```

To get more information about the **`simplify`** function, type:

```
help sym/simplify
```

```
 SIMPLIFY Symbolic simplification.
    SIMPLIFY(S) simplifies each element of the symbolic matrix S.

    SIMPLIFY(S,N) or, equivalently, SIMPLIFY(S,'Steps',N),
    searches for a simplification in N steps. The default value of N is 1.

    SIMPLIFY(S,'Seconds',T) aborts the search for a simpler version
    at the next end of a simplification step after T seconds. The results
    will depend on the speed and system load of your computer and may
    not be reproducible.
```

```
SIMPLIFY(S,'IgnoreAnalyticConstraints',VAL) controls the level of
mathematical rigor to use on the analytical constraints while simplifying
(branch cuts, division by zero, etc). The options for VAL are TRUE or
FALSE. Specify TRUE to relax the level of mathematical rigor
in the rewriting process. The default is FALSE.

SIMPLIFY(S, 'Criterion', 'preferReal')
discourages simplify from returning results
containing complex numbers.

SIMPLIFY(S, 'All', true)
returns a column vector of aquivalent results. Typically used in
combination with 'Steps', N.

Examples:
   syms x c alpha beta

   simplify(sin(x)^2 + cos(x)^2)  returns  1.
   simplify(exp(c*log(sqrt(alpha+beta))))  returns  (alpha + beta)^(c/2).
   simplify(sqrt(x^2))  returns  sqrt(x^2),
   simplify(sqrt(x^2),'IgnoreAnalyticConstraints',true)  returns  x.
   u = symunit;
   simplify(u.m+u.mm) returns  (1001/1000)*[m].

See also SYMUNIT, SYM/FACTOR, SYM/EXPAND, SYM/COLLECT.
```

Question 2 (2 marks)

Write the symbolic expression of the following functions (you can use the symbolic variables **`t`** and **`s`** defined previously):

1. $f_1(t) = \delta(t)$ ($\delta(t)$ is the dirac function, use **`help dirac`** for more information)
2. $f_2(t) = t^2 + 4t + 1$
3. $f_3(t) = te^{-t} + 0.5e^{-t} + e^{-3t}$
4. $f_4(t) = \sin(2t) + \cos^2(t)$
5. $f_5(t) = e^{-t}\sin(5t + \frac{\pi}{3}) + t^2e^{-2t}$

```
f1 = []
f2 = []
f3 = []
f4 = []
f5 = []
```

```
f1 =

     []

f2 =

     []

f3 =

     []

f4 =

     []

f5 =

     []
```

Question 3 (2 marks)

Use **MATLAB** to compute the Laplace transform of the functions defined in *Question 2*. Use **simplify** to simplify the symbolic expression if needed.

```
F1 = []
F2 = []
F3 = []
F4 = []
F5 = []
```

```
F1 =

     []

F2 =

     []

F3 =

     []

F4 =

     []

F5 =

     []
```

3.2. Computing inverse Laplace transforms

The function **ilaplace** computes the inverse Laplace transform. Consider the Laplace transform **F** computed earlier. We can compute the inverse Laplace transform (note that the **t** and **s** are already defined as symbols)

Example

```
ilaplace(F, s, t);
```

Question 4 (2 marks)

Use **MATLAB** to calculate the inverse Laplace transform of the following expressions. Use **simplify** to simplify the symbolic expression if needed.

1. $G_1(s) = \frac{1}{s}$
2. $G_2(s) = \frac{1}{s(s+1)}$
3. $G_3(s) = \frac{5}{s^2+25}$
4. $G_4(s) = \frac{10s+1}{s^2+2s+10}$
5. $G_5(s) = \frac{2(s+1)(s^2+4)}{(s^2+4s+1)(s^2+9)}$

```
G1 = [];
G2 = [];
G3 = [];
G4 = [];
G5 = [];

g1 = []
g2 = []
g3 = []
g4 = []
g5 = []
```

```
g1 =

     []

g2 =

     []

g3 =

     []

g4 =

     []

g5 =

     []
```